SOF Diagnostic Protocol

A Report Specification for Sectorized Observable Analysis

WuJun Chen
Independent Researcher
2026

This paper is Part XII of the RIME program. Paper VIII defines the static Sectorized Observable Framework (SOF), Paper IX studies observable deformations, Paper X introduces the Universal Observable Pipeline and the SOF Registry, and Paper XI turns wall records into an observable classification layer. Paper XII asks how SOF is deployed: how does one produce a reusable SOF Report for a concrete system?

Abstract

Problem. Papers VIII–XI develop the SOF object language, deformation geometry, registry evidence, and wall-record taxonomy. These layers answer what an SOF is, why different species can share one observable pipeline, and when observable walls or signatures appear. What remains is the methodological question: how does one actually use SOF on a new system? A useful answer cannot be another abstract theorem alone. It must produce an artifact that a reader can run, inspect, and adapt. A shared report grammar also prepares artifacts for later aligned comparison without defining that comparison here.
Approach. We formulate the **SOF Diagnostic Protocol** as a deployable workflow whose standard output is a **SOF Diagnostic Report**. The protocol's report standard is: SOF diagnostics = sectorized observable reports. Its deployment slogan is: **No weights required. Only observables.** A SOF Report records sectorization, observable family, support matrix, bridge matrix, repair matrix, wall record, claim status, and failure modes. The diagnostic pipeline is:

$$\begin{aligned}&\text{input system} \to \text{sectorization} \to \text{observable family}\\&\qquad\to \text{sector audit} \to \text{observable shadows} \to \text{SOF Report.}\end{aligned}$$

Results. The paper gives a first report schema and a set of reproducible case studies. In a transformer-like activation SOF, token sectors induced by FFN activation counts yield non-trivial cross-sector accessibility. In a real pretrained Qwen audit, attention heads themselves induce information sectorizations: one head collapses all tokens into a global sector, another produces a coarse three-group partition, another supplies four natural SOF sectors, and another yields a dispersed thirteen-group partition. This is a white-box SOF diagnostic because internal attention matrices are available. A second, behavioral regime treats prompt protocols and task classes as probe sectors for API-only models and records Structural, Behavioral, and Failure observables without claiming access to internal mechanisms. A Mixture-of-Experts audit shows that top-2 routing naturally induces six expert-pair sectors. In a synthetic transformer batch sweep, increasing token count increases the number of sector pairs under analysis while preserving the qualitative repair/freeze pattern. In a diffusion toy audit, diffusion time is a natural deformation parameter: forward noise creates a sector split, while reverse denoising is the repair direction. A dynamic maze gives 24 split crossings and 24 reverse merge/repair crossings. A recommender audit detects 12 of 16 user-cluster/item-cluster pairs as structural coverage dead zones. A black-box LLM audit adds an API-only behavioral report with Structural, Behavioral, and Failure observable families while marking prompt protocols as probe sectors and assigning `claim_status: diagnostic` with a weak behavioral qualification rather than claiming a strict SOF

realization. Schema, few-shot, and tool repair are recorded separately as protocol transitions. Finally, five constructed failure cases show when the SOF Report is not applicable, uninformative, degenerate, or not useful, while a separate API-infrastructure boundary distinguishes provider failure from model behavior.

Implications. Paper XII positions SOF as a methodology rather than a new universal theory. SOF is deployable when it produces a claim-status-aware SOF Report. Such a report can be positive, negative, or degenerate; the ability to return failure modes is part of the method. A report is not an absolute image of its source system: it is derived from a declared SOF realization and reporting specification. White-box and behavioral reports show that SOF is an observable framework, not a weight framework. The report format also supplies a stable contract for future diagnostic tooling. It standardizes the artifact, not the scientific adequacy of a domain realization, which still requires domain-specific justification.

---

1 Introduction

The first eleven papers of the RIME program build a sequence of increasingly general observable layers. Papers I–III establish the Rubik laboratory. Papers IV–VII develop collision geometry, accessibility repair, commutativity walls, and generic completion. Papers VIII–XI then extract the SOF program: the object layer, deformation layer, registry layer, and wall-record classification layer [3, 2, 4, 1].

Paper XII changes the question. It does not ask for another universal SOF theorem. It asks how SOF becomes usable.

In short, Papers VIII–XI ask what SOF objects are, why their observables matter, and when their dynamics change; Paper XII asks how to use them.

The core methodological claim is: **SOF diagnostics are sectorized observable reports.**

The standard artifact is the **SOF Report**. Like a test report, coverage report, profiling report, or static-analysis report, a SOF Report is meant to be produced by a workflow. It is not merely explanatory prose. It records what sectorization was used, which observables were extracted, what support or repair structure was found, which wall or trajectory diagnostics are present, and what claim-status boundary applies. Model cards and internal algorithmic audit frameworks provide related precedents for structured, scope-aware reporting [9, 12].

This is why Paper XII is a methods paper. SOF becomes deployable only when the abstract object language can be converted into a reproducible report.

The sharpest deployment statement is: **SOF diagnostics do not require access to internal mechanisms; they require observable structure.**

---

2 SOF Report Specification

Definition 2.1 (SOF Report Specification).

The **SOF Report Specification (SOFRS) v1.0** is the versioned machine-readable contract for the fixed-format output of the SOF Diagnostic Protocol. A conforming report describes one named system, sectorization, observable family, and claim status. Its eight required diagnostic fields are: Each field has a specific role.

Field	Meaning
Sectorization	origin, sector dimensions or probe classes, construction rule, and strict/probe realization status
Observable Family	operators, matrices, kernels, transitions, or registered analogues used for the audit
Support Matrix	direct cross-sector support, typically an R_1-type shadow when available
Bridge Matrix	typed length-two, word, Lie/commutator, routing, or explicitly labeled behavioral bridge analogue; the bridge semantics must be named
Repair Matrix	observed frozen-to-active pairs, expert reactivations, or protocol failure-to-success transitions, with their repair layer or step; this field is descriptive rather than an action recommendation
Wall Record	jumps, terminal boundaries, plateau events, collision loci, and, when a deformation exists, the associated trajectory summary
Claim Status	one controlled claim class from the vocabulary below
Failure Modes	applicability limitations, degeneracy, unavailable diagnostics, evaluator boundaries, or other warnings about interpreting the report

The controlled `Claim Status` vocabulary is `theorem`, `evidence`, `diagnostic`, `proxy_only`, `boundary`, `failure`, and `negative_control`. Human-readable qualifications belong in `claim_note`, not in the controlled status value.

2.1 Protocol Stack Boundary

SOFRS is the **diagnostic language** of the protocol stack. Its unit is one named system, run, snapshot, or parameterized trajectory:

$$\text{system or run} \longmapsto \mathcal{R}_{\mathrm{SOF}}.$$

It does not align two independent reports, compute a cross-report difference, interpret that difference as an action consequence, or select a policy. Those operations belong to later layers:

Paper	Input and operation	Artifact
XII	describe one system, run, snapshot, or internal trajectory	`.sofreport`
XIII	align two reports and compute an Audit Signature	`.sofaudit`
XIV	interpret signature coordinates and derive candidate actions	`.sofaction`

A `Wall Record` may contain before/after values or a trajectory internal to one report when a deformation path is part of the audited system. This is not the same object as Paper XIII alignment between two separately emitted reports.

Likewise, `Repair Matrix` records repair that was observed in the audited data. It does not prescribe `repair`, `monitor`, `preserve`, `contain`, or `validate` actions. Those are downstream Action Semantics.

2.2 Observable Family Registry

For deployable diagnostics, the observable family should be classified by what it measures rather than stored as an undifferentiated list. Paper XII uses three primary families:

Family	Typical observables	Role in the report
Structural observables	JSON validity, XML validity, schema consistency, valid tool-call emission	measure whether an output satisfies an externally specified structure or protocol
Behavioral observables	instruction following, task completion, task-scoped groundedness, preference or protocol alignment	measure whether the response preserves the requested task and behavioral constraint
Failure observables	refusal, grounded-answer failure, format collapse, prompt-injection failure	record named externally visible system events under a specified evaluator

Repair is not a fourth observable family. It is a transition recorded in the **Repair Matrix**, for example schema repair, few-shot repair, tool repair, or private-expert reactivation. These classes are report fields, not universal invariants. In particular, task-scoped groundedness is not a general hallucination detector, and protocol-level repair is not identified with Lie-depth D-repair. The report must name the evaluator, task scope, and probe/evaluation protocol for every behavioral or failure observable.

Failure observables and the top-level `Failure Modes` field are distinct. Failure observables are measured outputs of the audited system. `Failure Modes` records limitations of the report, realization, evaluator, or protocol itself.

The fields are fixed even when their values are negative or unavailable. A static transformer audit records `Wall Record: not applicable; no deformation path supplied`. A diffusion audit records its trajectory inside `Wall Record`. A failure case still emits all eight fields, with an explicit failure status. The point is a stable report grammar that places unlike systems in a common syntax. Pairwise comparability still requires the explicit sector and observable alignment introduced in Paper XIII.

Every artifact also carries `sofrs_version: "1.0"`. Human-readable qualification belongs in the optional `claim_note` field rather than in the controlled status value. The machine-readable artifact uses the `.sofreport` extension. The intended convention is one report per experiment or named system, for example `maze.sofreport`, `transformer.sofreport`, `diffusion.sofreport`, or `qwen.sofreport`.

Implementations should additionally provide stable metadata such as `report_id`, `system`, applicability or realization status, and provenance for data, model, evaluator, thresholds, and code version. These are metadata around the eight diagnostic fields, not a ninth diagnostic layer. They are recommended for reproducibility and become essential when Paper XIII later references and aligns reports.

2.3 Envelope Validity and Protocol Admission

SOFRS v1.0 intentionally separates two validation questions. **Envelope validity** means that an artifact satisfies the frozen JSON Schema: the eight fields are present, `claim_status` belongs to the controlled vocabulary, and superseded or downstream top-level fields are absent. Envelope validity alone does not establish that the scientific protocol has been followed.

Paper XII protocol admission is the stronger profile. It additionally requires a named `system`, stable `report_id`, explicit `claim_note`, nonempty failure boundary, and a wall result or explanation. A Level III behavioral report must also identify its source interface, evaluator, evaluator protocol, and evaluator scope. If Support, Bridge, and Repair are all unavailable, a boundary or failure report must explicitly name the unavailable diagnostics and the reason. Thus:

Formally,

$$\text{SOFRS envelope-valid} \not\Longrightarrow \text{Paper XII protocol-admissible.}$$

The frozen schema, independent admission profile, and stronger validator are listed as Artifacts B1–B4 in Appendix B.

---

3 SOF Report Protocol

The operational pipeline is:

$$\begin{aligned} \text{input} &\to \text{compatible sectorization} \to \text{observable family} \\ &\to \text{support audit} \to \text{bridge audit} \to \text{repair audit} \\ &\to \text{wall record} \to \text{SOF Diagnostic Report.} \end{aligned}$$

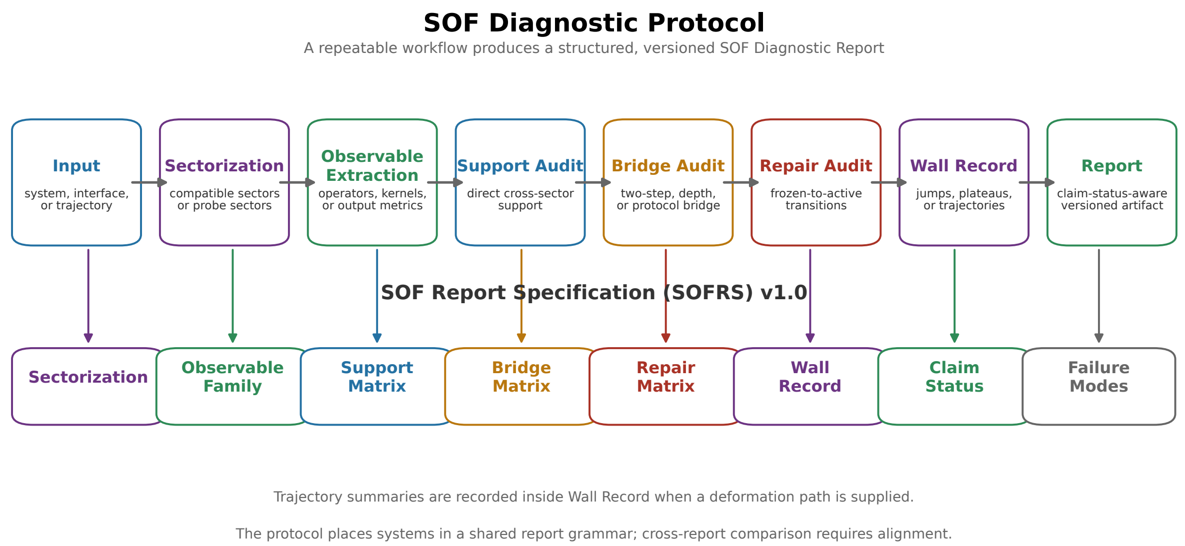

Figure 1. SOF Diagnostic Protocol. Eight protocol stages transform a named input into a fixed-format SOF Diagnostic Report. Trajectory summaries belong to Wall Record when a deformation path is supplied; the report places systems in a common output grammar but does not itself perform cross-report alignment or comparison.

The protocol is executed in the following order:

1. **Input:** name the system, interface, state space, or deformation path.
2. **Sectorization:** construct compatible sectors or label probe sectors.
3. **Observable extraction:** name the operators, kernels, outputs, or metrics.
4. **Support audit:** compute direct cross-sector support.
5. **Bridge audit:** compute a named word, Lie, length-two, routing, or behavioral bridge and declare its semantics.
6. **Repair audit:** record observed frozen-to-active transitions in the Repair Matrix without inferring an intervention command.
7. **Wall record:** record jumps and trajectories when a path has been supplied.

8. **Report:** emit the eight fields with claim status and failure modes.

The pipeline assumes neither a representation-theoretic origin nor a universal source of sectors. In the Rubik laboratory, sectors come from joint spectral geometry. In control systems, they may come from controllability flags. In finite Markov systems, they may come from state partitions or absorbing classes. In neural systems, they may come from activation patterns, attention heads, token groups, residual subspaces, or expert routing.

The common requirement is a stable information decomposition: a sectorization in which information flow, influence, reachability, hitting, activation, or propagation can be measured.

3.1 Domain Realization Responsibility

SOFRS standardizes the diagnostic artifact; it does not standardize the native scientific meaning of every possible sectorization or observable family. A domain application therefore has two distinct responsibilities:

Responsibility	What must be supplied
SOF protocol	field semantics, artifact versioning, validation, claim status, failure boundaries, and reproducibility requirements
Domain realization	source-to-sector construction, observable selection, thresholds, native constraints, and domain interpretation
Joint analysis	applicability level, realization justification, known blind spots, and the scope of conclusions supported by the report

Principle (Domain Realization Principle). A conforming SOF Report records a declared realization; schema validity and protocol admission do not by themselves establish that the realization is scientifically adequate for its source domain. That adequacy requires domain-specific justification and may require subject-matter expertise.

This division is intentional. SOF supplies a common diagnostic language and infrastructure, while domain specialists determine which decompositions and measurements preserve the distinctions that matter in their systems. A weak realization can produce a formally valid but scientifically uninformative report, and Failure Modes must say so.

---

4 Report Relativity

A SOF Diagnostic Report is never absolute. It is always relative to the declared sectorization, observable family, and reporting specification.

The report is therefore not obtained directly from a source system S. One first chooses and justifies a realization

$$\mathcal{F}_\eta = \mathrm{Realize}_\eta(S) = (V_\eta, \{Q_i^\eta\}, \mathcal{X}_\eta),$$

where η records the realization choices: retained finite space, sectorization, observable extraction, truncation, thresholds, and any other source-to-SOF decisions. A reporting specification Θ_{rep} then determines how the realized shadows are serialized, aggregated, qualified, and assigned claim status:

$$\mathcal{R}_{\eta,\Theta_{\mathrm{rep}}}(S) = \mathrm{Report}_{\Theta_{\mathrm{rep}}}(\mathcal{F}_\eta).$$

The complete chain is

$$S \xrightarrow{\text{Realize}_\eta} \mathcal{F}_\eta \xrightarrow{\text{Report}_{\Theta_{\text{rep}}}} \mathcal{R}_{\eta,\Theta_{\text{rep}}}(S).$$

This distinguishes three objects that must not be identified:

Layer	Object	Role
Source	S	the underlying physical, algebraic, computational, or behavioral system
Realization	$\mathcal{F}_\eta$	the chosen finite SOF object or diagnostic realization
Report	$\mathcal{R}_{\eta,\Theta_{\text{rep}}}$	the versioned protocol artifact derived from that realization

The SOF Report is a genuine object of the diagnostic protocol, but it is not the source object itself and it is not an intrinsic, unique image of that source. It is a derived epistemic artifact.

The conceptual division is concise:

> **Sectorization determines the ontology of the report; observable families determine its epistemology.**

Sectorization declares which parts or information classes exist for the analysis. The observable family declares which relations, transitions, or responses can be detected between those parts. The reporting specification declares tolerances, depth semantics, aggregation, evaluator provenance, and claim boundaries. Altering any of these may alter the report without altering the underlying source system.

Principle (Report Relativity Principle). Two SOF Reports derived from the same underlying system need not coincide, because realization choices may alter the measured sectors, observable families, reporting semantics, or all three. Consequently, equality of reports is not a primitive scientific notion; comparability requires explicit alignment.

For two admissible realizations of the same source,

$$\mathcal{R}_1 = \text{Report}_{\Theta_1}(\text{Realize}_{\eta_1}(S)), \qquad \mathcal{R}_2 = \text{Report}_{\Theta_2}(\text{Realize}_{\eta_2}(S)),$$

one may have $\mathcal{R}_1 \neq \mathcal{R}_2$ even though the source S is unchanged. The discrepancy may come from $\eta_1 \neq \eta_2$, from $\Theta_1 \neq \Theta_2$, or from both. A report difference is therefore not, by itself, a system difference or a defect.

Three notions should be kept separate:

1. **Literal report equality:** two serialized artifacts have the same fields and values. This is reproducibility at fixed realization and specification.
2. **Aligned report comparability:** sectors, observables, depth semantics, and normalization are related by explicit maps. This is the object of Paper XIII.
3. **Realization invariance:** a statement survives a declared class of admissible realizations. This is a stronger theorem-level property and must be proved rather than assumed.

The need for Paper XIII now follows directly. If reports were absolute and canonical, alignment would be bookkeeping. Because reports are realization-relative, alignment is a mathematical prerequisite for comparison:

$$(\mathcal{R}_1, \mathcal{R}_2) \not\mapsto \Delta \quad \text{without} \quad (\Phi_{\text{sec}}, \Phi_{\text{obs}}, \Theta_{\text{cmp}}).$$

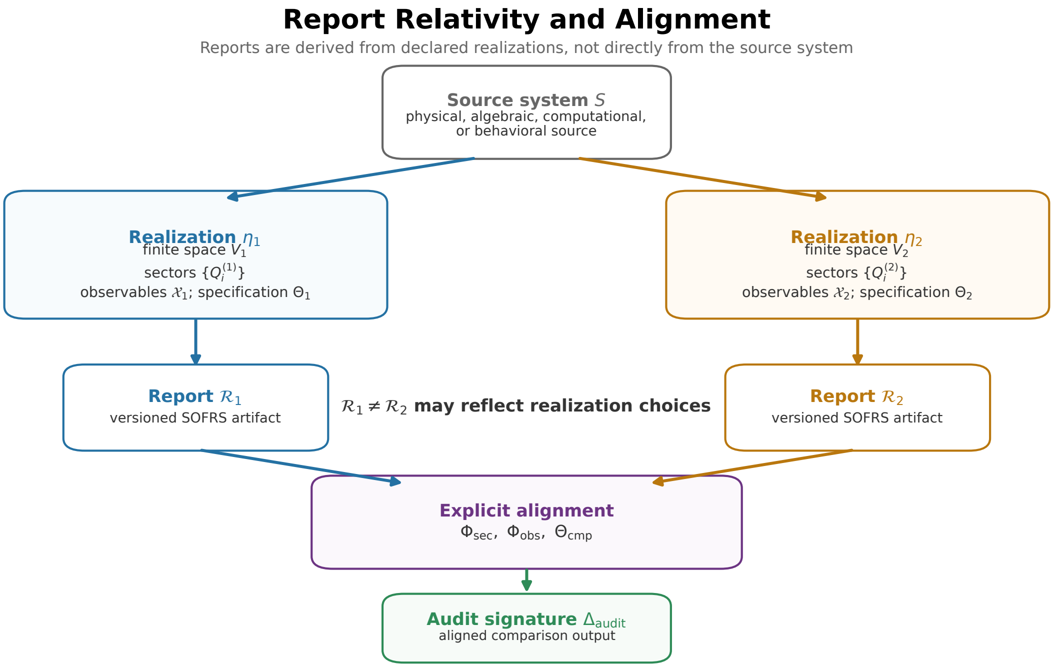

Figure 2. Report Relativity and Alignment. One source system may admit multiple declared realizations and therefore multiple versioned SOF Reports. A report difference is not by itself a source-system difference: sector, observable, and comparison alignment are required before an Audit Signature is formed.

4.1 Quantum Probe Example

The focused quantum controls (Artifacts B17–B18) exhibit this relativity on one quantum gate-family comparison.

Realization	Sectorization	Observable family	Observed T sensitivity
gate-log	computational-basis projectors	skew-Hermitian gate logarithms	none; the Clifford and Universal $R_1/R_2/D$ shadows coincide
state trajectory	coarse STAB/MAGIC state classes	empirical class-transition observable	skew signal $0 \to 0.084$; off-diagonal support $0 \to 2$

The first realization is blind to the T/S diagonal-phase distinction. The second converts magic-state production into complete off-diagonal support in a two-state coarse graining. STAB and MAGIC are nonlinear state classes rather than orthogonal subspaces of the qubit Hilbert space, so the second construction is a trajectory-induced Markov realization, not a strict projector sectorization of $\mathbb{C}^2$.

This example does not say that changing observables and changing sectors are mathematically identical. They are two independent ways to redesign the probe: one changes what counts as a part of the report, while the other changes what the report can detect. Both modify the realized SOF object and must be declared.

The freedom is constrained rather than arbitrary. Every realization must state its sector origin, observable construction, truncation, evaluator, thresholds, and failure modes. SOF permits multiple realizations; it does not permit hidden realization choices.

5 SOF Applicability Hierarchy

The existence of a sector-like vocabulary is not by itself enough to justify an SOF claim. Paper XII therefore introduces the **SOF Applicability Hierarchy**, which measures the distance between a proposed application and a formal SOF realization. It is a methodological hierarchy, not a theorem that the four levels form a categorical filtration.

The levels classify the justification used in a particular analysis, not a species permanently. A realizational application produces a definitional SOF target once its map has been fixed, but a claim about the legitimacy or non-uniqueness of the source-to-object map remains Level II. The same source may therefore appear at different levels under different analyses. Different realizations of the same source system need not be equivalent. They may use different finite spaces, sector resolutions, retained subspaces, or observable families, and equivalence must be established rather than assumed.

Level	Minimum requirement	Representative examples	Permitted use
I: Definitional	an explicit finite $\mathcal{F} = (V, \{Q_i\}, \mathcal{X})$ satisfying the SOF axioms	strict finite Rubik, quantum, Markov, graph, NCG, control, and PDE realizations	formal SOF constructions and qualified theorem, evidence, or diagnostic claims
II: Realizational	a reproducible source-to-SOF map specifying finite space, sector origin, observable extraction, truncation, and non-uniqueness	transformer activations, attention heads, MoE routing, recommendation graphs, and visible hidden states	construction-level evidence and white-box SOF Reports
III: Diagnostic	stable probe sectors, measurable outputs, evaluator provenance, and an explicit claim boundary; no strict projector realization is required	API-only language models, closed models, and external behavioral interfaces	Behavioral or API-level SOF Reports only
IV: Analogical	only a structural or linguistic resemblance to sectors, walls, repair, or accessibility	candidate agent, economic, biological, or social analogies	heuristic discussion only

The four levels have the following admission rules.

1. **Definitional applicability** requires a complete finite orthogonal decomposition $\sum_i Q_i = I$ and a named observable family on V.

2. **Realizational applicability** additionally requires an auditable map from the source system to the SOF data. For example, the Qwen case uses a strict four-sector realization on a stated 40-token retained subspace rather than silently treating a filtered partition of all 45 tokens as complete.

3. **Diagnostic applicability** does not require an internal SOF object, but it does require stable probe sectors, measurable outputs, evaluator and protocol provenance, and failure reporting. Prompt labels alone are not enough.

4. **Analogical applicability** cannot by itself produce a conforming SOFRS artifact or enter the SOF Registry as a realized species. It may motivate a future sectorization or diagnostic protocol, but its language must remain explicitly heuristic.

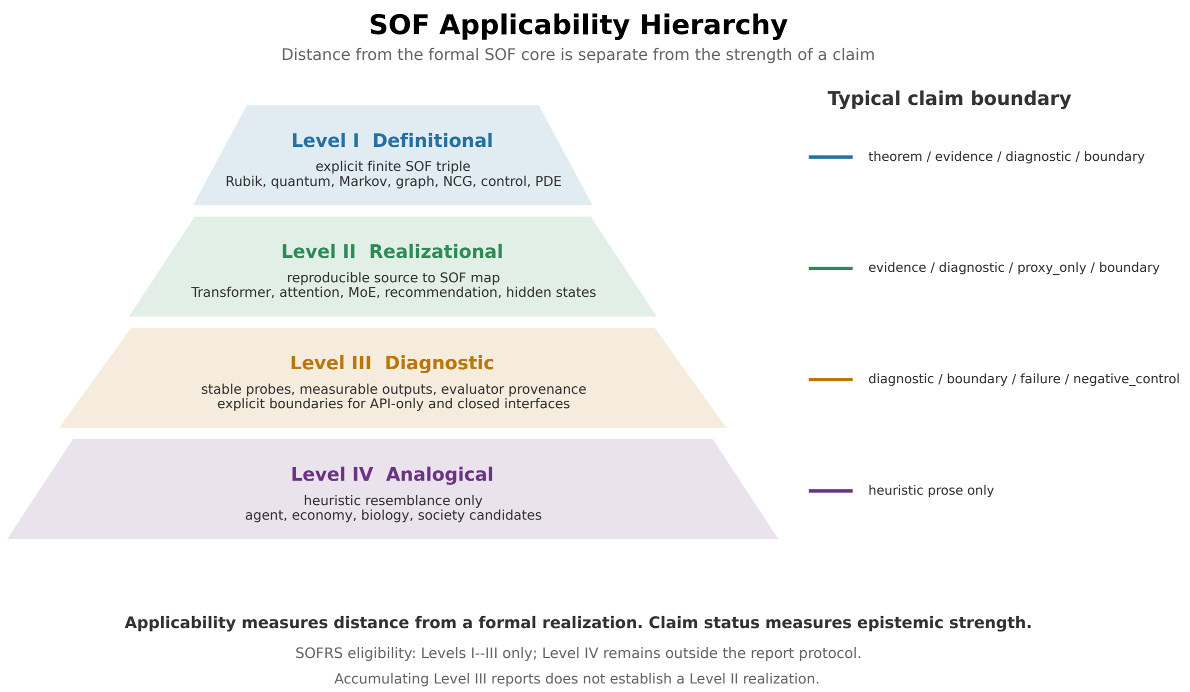

Figure 3. SOF Applicability Hierarchy. Applicability measures distance from the formal SOF core: explicit SOF objects, reproducible source realizations, report-level probe diagnostics, and heuristic analogies. Claim status is an independent epistemic axis and must not be inferred from applicability level.

Principle 1 (Applicability Monotonicity Principle). Applicability may be weakened conservatively by forgetting structure or restricting attention to a report-level interface:

$$\text{Level I} \longrightarrow \text{Level II} \longrightarrow \text{Level III}.$$

The reverse direction is not automatic. In particular, Level III never upgrades to Level II merely by accumulating diagnostic reports. Promotion requires a new realizational construction together with an explicit finite space, sectorization, observable family, and justification of the source-to-SOF map. Likewise, repeated analogies do not establish Level III without a realizable diagnostic protocol.

SOFRS Eligibility. Only Level I–III applications are eligible to produce conforming SOFRS artifacts. Level I and Level II reports are attached to formal or realized SOF data; Level III reports are explicitly diagnostic or behavioral. Level IV applications remain outside the report protocol until a realizable diagnostic pipeline with stable probes, measurable outputs, evaluator provenance, and failure boundaries has been defined.

Applicability and claim status are independent coordinates:

Applicability level	Typically admissible `claim_status` values
Definitional	`theorem`, `evidence`, `diagnostic`, `boundary`
Realizational	`evidence`, `diagnostic`, `proxy_only`, `boundary`
Diagnostic	`diagnostic`, `boundary`, `failure`, `negative_control`
Analogical	outside the formal claim-status system; label as heuristic prose

In particular, definitional applicability does not imply a theorem, and a diagnostic report does not imply an internal realization. **Applicability measures distance from a formal**

SOF realization; claim status measures the strength of what is known at that distance.

SOFRS v1.0 records this distinction through the Sectorization, Claim Status, Claim Note, and Failure Modes fields. Applicability remains report metadata rather than a required v1.0 diagnostic field.

---

6 White-Box and Behavioral SOF Diagnostics

SOF deployment separates into two diagnostic regimes. The distinction concerns what is observable, not whether the source system is neural.

Definition 6.1 (White-Box SOF Diagnostic)**.**

A SOF Report is **white-box** when its sectorization and observable family are extracted from an explicitly specified finite internal realization, such as representations, weights, activations, state operators, controllability flags, mesh operators, or known transition matrices.

Definition 6.2 (Behavioral SOF Diagnostic)**.**

A SOF Report is **behavioral** when its sectors are induced from externally observable interfaces and its observable family is computed from input–output behavior. Prompt protocols and task classes may serve as **probe sectors**. Unless these probe sectors are separately realized as orthogonal projectors on a specified finite space, the report is a weak behavioral diagnostic rather than an object of the strict category $\mathsf{SOF}_{\mathrm{str}}$.

Regime	Sector source	Observable source	Claim boundary
White-box	internal representations, activations, operators, or known state decompositions	matrices, kernels, weights, transitions, and support blocks	strict or represented SOF diagnostic only when the finite realization is explicit
Behavioral	externally visible protocols, task classes, response classes, or interface states	format, semantic, operational, latency, tool, and repair outcomes	weak behavioral diagnostic unless a strict projector realization is supplied

The methodological point is that **SOF is an observable framework, not a weight framework.**

Weight access is sufficient for some SOF realizations, but it is not necessary for a SOF Report. What is necessary is a stable information decomposition, an explicit observable family, and a claim status that says whether the report is strict, represented, proxy-only, or behavioral.

Principle 2 (Black-Box SOF Diagnostic Principle). A white-box realization is sufficient but not necessary for SOF diagnostics. If a compatible sectorization or stable probe-sector decomposition is available and observable outputs are measurable, then a claim-status-aware SOF Report can be produced without access to the underlying internal representation.

The conclusion is report-level, not mechanism-level. The principle permits external structural, behavioral, failure, wall, and repair diagnostics; it does not reconstruct hidden weights, certify a strict object of $\mathsf{SOF}_{\mathrm{str}}$, or identify the internal cause of an observed transition.

6.1 Attention Heads as Natural Sectorizers

A central application domain is neural-system diagnosis. Transformer-style systems already contain mechanisms that partition information: activation patterns, attention heads, token groups, residual-stream directions, and expert-routing decisions [13, 5].

The central observation from the Qwen audit is that **attention heads naturally induce information sectorizations.**

This statement is more important than any one accessibility percentage. A pretrained attention head can itself define the sectorization by grouping tokens with common top-attention targets. No synthetic generator or external partition is needed.

The Qwen attention audit (Artifact B7) uses the revision-pinned `Qwen/Qwen2.5-0.5B-Instruct` model [11]. The recorded configuration audits layer 12 and groups tokens by their strongest attention target. For the 45-token probe, the layer has 14 heads. The observed head diversity is:

The reference artifact records the exact model/tokenizer commit, Transformers and PyTorch versions, device, dtype, input text, layer, head, and filtering parameters. Cache location is a command-line option and is not embedded as a machine-specific dependency.

Head	Groups	Interpretation
Head 3	1 group	global attention: all tokens attend to one target
Head 1	3 groups, sizes `[38,4,3]`	coarse three-way token clustering
Head 6	9 target groups, 4 sectors after filtering groups of size at least 2	natural SOF sectorization used for the token-space audit
Head 13	13 groups	dispersed attention partition

Thus one pretrained layer supplies multiple sectorization granularities: global, coarse, intermediate, and dispersed. SOF does not impose these partitions; it records them and asks what observable shadows they carry.

Using the Head 6 sectors and the layer's attention matrices as observables produces the complete eight-field artifact reported in Case Study A. The main significance is structural: attention heads provide sectorizations at different resolutions before any SOF machinery is applied.

6.2 Behavioral Walls

Behavioral diagnostics inherit the wall language of Papers IX and XI only after a deformation variable has been chosen. A collection of prompts is not yet a wall geometry. A parameterized protocol path $p(t)$ can, however, pull behavioral observables back to trajectories and wall records.

Behavioral wall	Example deformation variable	Observable event
Instruction-conflict wall	conflict strength, instruction order, or hierarchy position	instruction-following status changes
Refusal wall	task or policy-sensitive probe parameter	refusal status changes
Schema-collapse wall	schema complexity or constraint strength	format compliance drops or recovers
Context-saturation wall	context length or distractor density	correctness or consistency changes sharply
Prompt-injection wall	injection strength, location, or competing-instruction weight	control of the response shifts between prompt sectors

These are candidate **observable behavioral walls**, not claims about internal mechanisms. They belong to the SOF Report only when the path, evaluator, threshold, and claim status are explicit.

The same language gives a cautious behavioral repair pattern:

$$\text{zero-shot failure} \longrightarrow \text{demonstration bridge} \longrightarrow \text{few-shot behavioral repair.}$$

This is protocol-level repair, not Lie-depth D-repair. At the diagnostic level, instruction tuning and preference optimization, exemplified by InstructGPT [10], are naturally studied as observable deformations of prompt–response behavior rather than inferred from parameter count or treated merely as enlargement of the underlying SOF object.

---

7 Case Studies: Reading a SOF Report

A conforming SOF Report has a fixed eight-field envelope, but the payload is allowed to reflect the diagnostic regime. A strict finite realization may store binary support matrices. A topology-changing report may leave the Bridge Matrix undefined while recording a complete Wall Record. A behavioral report may store protocol-by-observable scores and must label its bridges as behavioral analogues. The common grammar supports cross-case reading without claiming that the native mechanisms are equivalent or already aligned.

A reader should inspect a report in the following order:

1. verify where the sectors came from;
2. identify which observables were actually measured;
3. read support before interpreting bridges or repairs;
4. check whether a deformation path exists before reading the Wall Record;
5. finish with Claim Status and Failure Modes before making a scientific claim.

The following three reports show the same specification under three different conditions: a static white-box realization, a topology-changing finite system, and an API-level behavioral diagnostic.

7.1 Case Study A: Static White-Box Report

The Qwen attention audit is a Level II realizational analysis whose retained token subspace carries an explicit finite Level I SOF target. Head 6 supplies four retained attention-target sectors, while all attention matrices in the audited layer form the observable family.

Field	Recorded value
Sectorization	four retained Head 6 attention-target sectors
Observable Family	fourteen attention matrices from the audited layer
Support Matrix	4×4 aggregated R_1; off-diagonal density 75.0%
Bridge Matrix	4×4 commutator R_2; off-diagonal density 75.0%
Repair Matrix	zero repaired pairs; three terminally frozen pairs
Wall Record	not applicable; one pretrained snapshot
Claim Status	`diagnostic`
Failure Modes	prompt-, layer-, and filter-dependent; singleton groups excluded; attention support is not a causal explanation

The important negative entries are part of the result. `Repair Matrix: 0` means that the audited commutator layer does not repair the three frozen pairs. `Wall Record: not applicable` means that no deformation variable was supplied; it does not mean that the report failed.

Figure 4 visualizes the versioned `qwen.sofreport` artifact and shows how the eight required fields coexist in one reader-facing specimen.

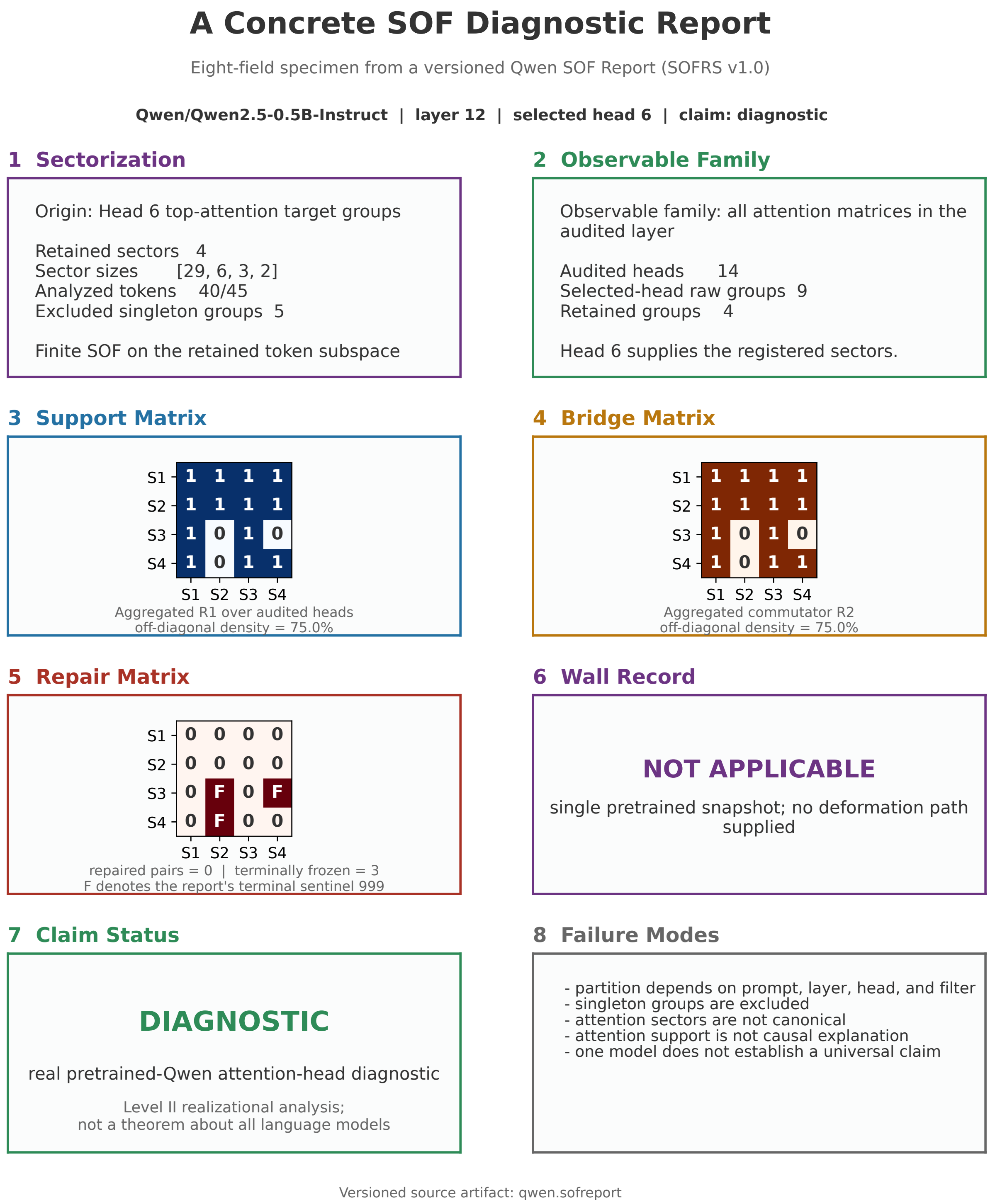

Figure 4. Concrete SOF Diagnostic Report specimen generated from the real pretrained-Qwen artifact. The eight panels instantiate the fixed SOFRS v1.0 fields: retained attention-head sectors, attention observables, support and bridge matrices, repair data, a static wall record, controlled claim status, and explicit failure modes.

7.2 Case Study B: Dynamic Wall Report

The maze report uses a native deformation path: doors are closed and then reopened. Its sectors are connected components of the current open-door graph, so the sectorization itself changes along the path.

Field	Recorded value
Sectorization	connected components of a 25-cell maze
Observable Family	open-door adjacency, connectivity, and frozen ordered pairs
Support Matrix	open: one component and zero frozen pairs; closed: 25 components and 600 frozen pairs
Bridge Matrix	null; not defined for this component-level report
Repair Matrix	24 reopening merges; frozen pairs decrease from 600 to 0
Wall Record	24 forward splits, 24 reverse merges, component path $1 \to 25 \to 1$
Claim Status	`diagnostic`
Failure Modes	sectors vary with door state; connectivity repair is not fixed-sector Lie-depth D-repair

This case shows why fields may be explicitly null. Inventing a Bridge Matrix would overstate the analysis. The informative objects are instead the changing sectorization, the support summary, the repair events, and the wall trajectory. The report remains conforming because the missing field is declared and its reason is recorded.

7.3 Case Study C: API-Level Behavioral Report

The NVIDIA NIM audit has no access to weights, activations, or hidden states. It is therefore a Level III behavioral analysis. Six prompt protocols crossed with three task classes define stable probe sectors, and measurable response properties define the observable family.

Field	Recorded value
Sectorization	six prompt protocols crossed with three task classes
Observable Family	Structural, Behavioral, and Failure observables
Support Matrix	protocol-by-observable score table covering completion, instruction following, groundedness, and provider success
Bridge Matrix	behavioral analogues for schema, few-shot, and tool changes
Repair Matrix	schema, few-shot, and tool failure-to-success transitions
Wall Record	not computed; the protocols form a discrete probe suite rather than a parameterized path
Claim Status	`diagnostic`; `strict_sof_realization = false`
Failure Modes	evaluator-, task-, provider-, and model-version scoped; protocol repair is not Lie-depth D-repair

Here `Support Matrix` is a declared protocol-by-observable score table rather than a projector-block matrix, and `Bridge Matrix` is explicitly marked as a behavioral analogue. The report therefore supports external behavioral claims but cannot reconstruct or certify an internal mechanism.

7.4 Cross-Case Reading Rule

Case	Diagnostic regime	Applicability	Correct reading
Qwen attention	white-box / realizational	Level II source map with a strict finite target	static support and bridge audit; no wall claim
Dynamic maze	explicit finite trajectory	Level I finite graph analysis	topology wall and connectivity repair; no component-level bridge claim
API-only LLM	behavioral / black-box	Level III diagnostic	external protocol outcomes only; no internal realization claim

The cases demonstrate the central reporting rule: **the same report grammar does not imply the same mechanism.**

SOFRS standardizes what must be disclosed. Claim Status and Failure Modes control how far each disclosed result may be interpreted. Cross-case rows remain reader-facing contrasts; a machine-generated pairwise difference requires the Paper XIII alignment object.

---

8 AI Systems

AI systems provide several distinct sector origins: activations, attention targets, expert routes, and diffusion-time feature partitions. The shared SOF Report does not erase those differences; it makes their observable consequences available in one syntax for later qualified comparison.

8.1 Transformer Activation SOF

The transformer activation audit (Artifact B5) builds a small transformer-like block. FFN activation-count clusters define token sectors, while attention and activation-similarity operators define the observable family. The reference report has three activation-count sectors, off-diagonal densities $R_1 = 58.3\%$ and $R_2 = 66.7\%$, two directly frozen pairs, two repaired pairs, and maximum finite depth 2.

The companion batch sweep (Artifact B6) tests whether the qualitative pattern survives a larger token partition. In the canonical `5 x 50` case it finds `frozen_R1=14`, `D_repaired=6`, `frozen_D=8`, and one permanently frozen sector. This is a robustness audit, not a theorem about all transformers.

8.2 Qwen Attention-Head SOF

The Qwen case supplies the white-box language-model instance in this domain. Its head diversity and realization construction are described in Section 6, while Case Study A and Figure 4 give the complete report. Its application-level role is to show that an internally visible model can supply multiple admissible sector granularities without making any one head partition canonical.

8.3 Mixture-of-Experts Routing SOF

MoE asks a different question: **why do experts specialize?** In the routing audit (Artifact B8), tokens sharing the same top-2 expert pair form a sector, and a routing-overlap kernel measures whether route sectors share an expert.

MoE diagnostic	Value
Natural routing sectors	6/6
Direct support	24/30 ordered pairs (80.0%)
Two-step routing repairs	6
Frozen routing-sector pairs	0
Maximum finite word depth	2

The diagnostic mapping runs from expert specialization to expert-route sectors and then to routing support and repair. The four-expert audit is a dense control; sparse specialization boundaries require larger expert pools, top-1 routing, capacity limits, fewer tokens, or expert dropout. Routing-word repair is not Lie-depth D-repair.

A second control (Artifact B9) models the load-balancing architecture associated with DeepSeekMoE and DeepSeek-V3 more closely [6, 7]. It separates fine-grained **private routed experts** from an always-active **shared baseline**. This separation is essential: putting the shared channel directly into private routing support would mask a dead private expert by making every token appear connected.

The private routing logits are deliberately imbalanced at the initial step. An auxiliary-loss-free routing bias is then updated by load: overloaded experts lose routing preference while underloaded experts gain it for the next top-k selection. In the finite control:

Bias-repair diagnostic	Value
Private experts / top-k / tokens	12/2/384
Initially active private experts	2/12
Initially frozen private experts	10/12
First routing repair step	18
Repaired private experts	10/10
Routing repair index	100.0%
Shared baseline	active for 384/384 tokens

This control gives a direct instance of **bias-driven routing repair**: a private expert is initially inactive and later crosses the routing threshold after a specified bias deformation. It is not Lie-depth D-repair. The correct relation is:

$$\text{load bias} \longrightarrow \text{private-routing repair trajectory} \longrightarrow \text{observable repair candidate and wall record.}$$

The control is inspired by the published architecture, not an audit of DeepSeek weights. In particular, the routing bias is a load-updated selection mechanism rather than an ordinary gradient-learned parameter in this report.

8.4 Diffusion Denoising SOF

Diffusion asks how an information decomposition changes along a native time parameter. In the denoising audit (Artifact B10), forward noise creates a probe-sector split at `t=11`, while reverse-time denoising crosses back between `t=11` and `t=10`. The experiment uses the forward/reverse denoising organization of diffusion models as background, not as a new diffusion theorem [8]. At the forward wall, $t = 11$ changes the probe from one sector to two at $\bar{\alpha} = 0.1296$, with six R_1 edges at the first split. Reverse denoising restores the clean endpoint signature.

The counterintuitive forward split is precisely why a trajectory-aware report is useful: noise does not merely erase sectors; under the chosen probe it can create an intermediate sectorization.

9 Dynamic Systems

Dynamic systems make wall and propagation semantics explicit. Here the main question is not expert specialization but **how topology or reachability changes**.

9.1 Dynamic Maze Wall Crossing

The maze audit (Artifact B11) is the topology-changing instance detailed in Case Study B. Its methodological role is to distinguish wall events from static frozen pairs: the initial connected component is already present, so the number of split events is one less than the final component count. Reverse door opening records connectivity repair rather than fixed-sector Lie-depth D-repair.

9.2 Kalman Reachability SOF

The control/PDE probe (Artifact B19) uses increments of the Kalman controllability flag as sectors. For the three-state chain, the Kalman ranks are `[1,2,3]`, the system is controllable, and the terminal sector first appears at word depth `2` from the input sector.

This report asks where control influence appears immediately and where it requires delayed propagation. The depth is a finite word/reachability depth, not automatically the commutator depth of Paper V.

9.3 PDE Interface Propagation SOF

The same computational control partitions a seven-point finite-difference grid into left, interface, and right sectors. The left-to-right block is not directly adjacent, but propagation through the interface appears at word depth `2`.

The native question is **how does influence cross a discretized interface?** The SOF Report records subdomain sectors, Laplacian support, the interface bridge, and propagation depth without claiming a general PDE theorem.

---

10 Industrial Diagnostics

Industrial reports emphasize actionable coverage, first-passage, and external behavior. Their value lies in identifying where a system cannot currently reach, comply, or recover before a more expensive evaluation is run.

10.1 Recommender Coverage SOF

Recommendation asks: **why do some items never appear?** In the recommender audit (Artifact B12), user clusters and item clusters are sectors and the bipartite interaction graph is the observable family. Direct user–item coverage is 4/16, leaving 12/16 recommendation dead zones. One targeted bridge reduces the dead-zone count from 12 to 10.

The 12/16 unreachable pairs are dead accessibility sectors for the audited collaborative-filtering propagation graph. This is an offline structural coverage signal. It identifies where an online experiment has no structural path to work with, but it does not replace ranking metrics, causal evaluation, or A/B testing.

10.2 Barrier-Finance SOF

The barrier-finance audit (Artifact B20) sectorizes a finite log-price grid into below-barrier and above-barrier regions. Drift and diffusion operators supply cross-barrier support, while a continuous-time Markov generator supplies a separate first-hitting-time diagnostic.

The reference audit reports $R_1 = 75.0\%$, $R_2 = 0.0\%$, no D-repair, and mean first-hit time 6.5915. First-hitting time is a native stochastic diagnostic; it is not identified with SOF depth D, and the case is not an option-pricing theorem.

10.3 API-Only LLM: API-Level SOF Report

The black-box LLM case asks the deployment question: **can SOF be used when weights and hidden states are unavailable?** The answer supplied by the Black-Box SOF Diagnostic Principle is yes at the report level, provided that stable probe sectors and measurable output observables are specified.

The black-box audit (Artifact B13) uses prompt protocols and task classes as probe sectors and emits an **API-level SOF Report**, a behavioral report subtype for a black-box language model. Its observable family is:

Family	Registered examples
Structural observables	JSON/XML validity, schema consistency, valid tool-call emission
Behavioral observables	instruction following, task completion, task-scoped groundedness, protocol consistency
Failure observables	refusal, grounded-answer failure, format collapse, prompt-injection failure when probed

Repair remains a report transition:

Repair candidate	Failure state	Protocol bridge	Success state
Schema repair	bare response violates the requested structure	explicit schema protocol	structurally valid response
Few-shot repair	bare response fails a closed task	demonstrated answer pattern	task completion
Tool repair	bare response omits the required call	tool-enabled protocol	valid tool call

The emitted report uses prompt protocols crossed with task classes as probe sectors, and it records schema, few-shot, and tool changes as behavioral bridge and repair analogues. With no parameterized prompt path, the Wall Record is not computed. Its controlled status is `diagnostic`, qualified by the claim note that this is a weak behavioral report rather than a strict SOF realization.

The deterministic fixture validates the evaluator and SOFRS envelope serialization. It is not evidence about any deployed API-only LLM until a real API audit is run and versioned. Black-box diagnostics expose observable behavior, not hidden mechanisms, and task-scoped groundedness is not a universal hallucination detector.

The recorded API-level report uses NVIDIA NIM with the model ID `meta/llama-3.1-8b-instruct` on 2026-07-11. All `18/18` protocol–task requests completed successfully. The report records:

Observable class	Recorded values
Structural	nonempty 100.0%, schema consistency 50.0%, valid tool call 16.7%
Behavioral	task completion 61.1%, instruction following 72.2%, task-scoped groundedness 83.3%, API success 100.0%
Failure	refusal 22.2%, grounded-answer failure 16.7%, format collapse 50.0%; prompt injection not measured
Repair	schema 2, few-shot 0, tool 1

These percentages describe this model/version, provider endpoint, evaluator, and prompt matrix only. They are not a ranking claim about language models. The versioned API artifact is included in the report collection (Artifact B15).

The response-level examples make the report more informative than a single accuracy number:

Task / protocol	Observed response event	SOF Report interpretation
arithmetic / JSON schema	`{"result": "4"}`	structural and behavioral success
grounded deadline / XML schema	correct plain-text answer rather than XML	semantic success with format collapse
weather / bare	explicit lack-of-tool-access response	externally visible refusal / no tool bridge
weather / tool enabled	`get_weather(city="Paris")`	valid tool repair
arithmetic / tool enabled	unsupported `add` function call	tool misuse on a non-tool task
grounded deadline / tool enabled	irrelevant weather call for New York	cross-sector protocol failure

These examples expose structural, behavioral, and failure observables separately. Only short normalized events are reported; long raw model prose and tool-call payloads are excluded.

10.4 Unified Deployment Table

System	Sector origin	Primary observable	Wall	Repair
Transformer model	activation clusters	attention / activation support	training path available	observed
Qwen	attention-target groups	attention support	not computed	none in the reference audit
MoE	expert routes	routing overlap / private loads	load-imbalance control	routing-word and bias-driven repair
Diffusion	time-indexed feature sectors	denoising trajectory	present	present
Maze	connectivity components	reachability	present	present
Kalman	controllability-flag increments	control propagation	optional path	word depth
PDE	mesh/interface partition	Laplacian propagation	optional path	interface bridge
RecSys	user–item graph clusters	structural coverage	intervention path	partial
Finance	barrier regions	cross-barrier support / first hit	candidate barrier path	none in the reference audit
API-only LLM	prompt/task probe sectors	response observables	candidate protocol path	protocol-level

The table is deliberately heterogeneous in native meaning. Its point is not that every repair is the same invariant. Its point is that every entry can be reported through the same sectorization–observable–wall–repair grammar with an explicit claim status. Every row emits the same SOFRS output grammar; the API-only audit is marked as an API-level SOF Report, with all `18/18` NVIDIA NIM requests successful in the reported audit.

Figure 5 summarizes the methodological distinction behind that deployment table. White-box and realizational analyses may use internal operators, kernels, routes, or explicit graph structure, whereas behavioral analyses use stable probe sectors and measurable outputs. They meet at the SOFRS contract, not at a claim that their internal mechanisms are equivalent. The Qwen, maze, and API-only LLM reports then instantiate Levels II, I, and III respectively while preserving the same report grammar.

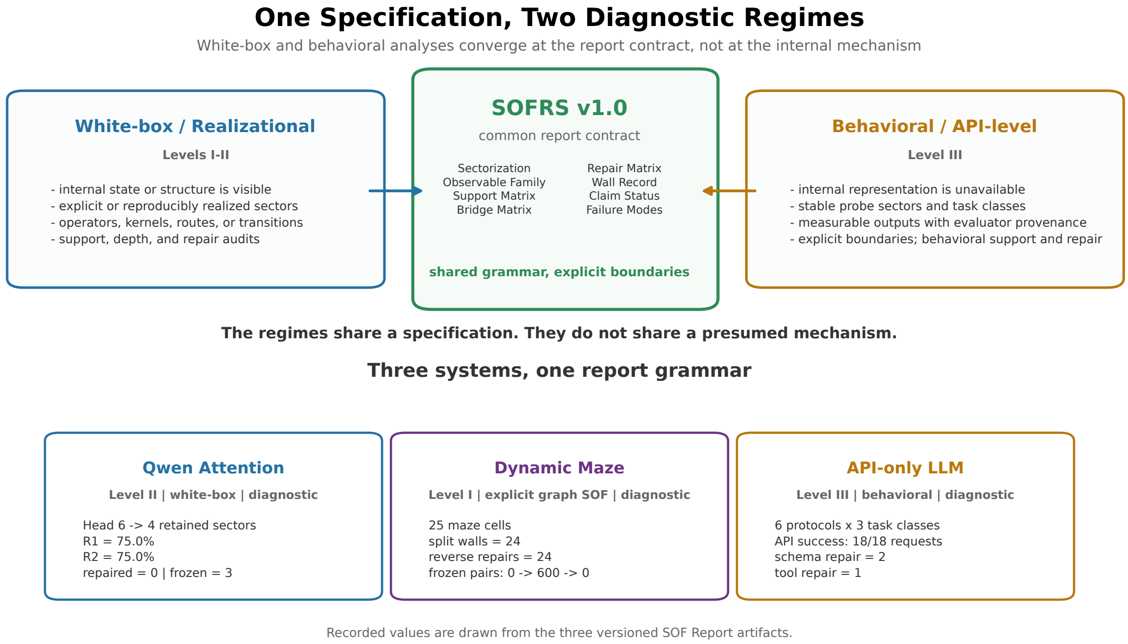

Figure 5. One specification, two diagnostic regimes. White-box and realizational analyses and behavioral API-level analyses enter SOFRS v1.0 through different observable interfaces. Qwen attention, dynamic-maze connectivity, and an API-only language model provide three claim-status-aware reports without being identified at the mechanism level.

11 Failure Modes and Applicability

A deployable method must say when it should not be used. The multi-system **validator fixture** (Artifact B14) contains five constructed structural boundary cases. It is envelope-valid so that validators can exercise all eight fields, but it is intentionally excluded from ordinary protocol admission. An API infrastructure boundary is listed separately because it can occur within a single behavioral report:

Case	Case interpretation	Diagnostic reason
Single sector	inapplicable	no cross-sector pair exists
Dense random all-to-all observables	no contrast	immediate full support destroys structure
Over-refined one-dimensional sectors	over-refined	sectorization is too fine to expose subspace structure
Commuting observables	no Lie bridge	the commutator-driven R_2 layer is absent
Sector-observable mismatch	probe mismatch	observables do not see the proposed sector interface
API infrastructure failure	infrastructure failure	provider or backend errors dominate the measurement

These interpretations are not `claim_status` values. An individual boundary construction can emit a conforming report with `claim_status: boundary` or `failure`, but the combined five-system fixture is not itself a normal single-system SOF Report.

The applicability conditions are therefore:

1. there must be at least two meaningful sectors;

2. the observable family must see the sector interfaces;
3. the report should not be all-to-all noise;
4. the sectorization should not be so fine that internal structure is erased;
5. if bridge or repair diagnostics are claimed, the observable family must support the corresponding layer.
6. an API-level report must separate provider success from model behavior and use `claim_status: failure` with an explanatory `claim_note` when infrastructure failures make the behavioral result inconclusive.

---

12 Machine-Readable Deployment Boundary

Paper XII requires a reproducible path from declared sectors and observables to the eight SOFRS fields, followed by envelope and protocol-admission validation. When a deformation path exists, its trajectory summary is recorded within Wall Record. The method does not require a particular software API or command-line interface.

The Paper XII boundary ends at report production and validation. Report alignment and normalized comparison belong to the downstream comparison layer; signature interpretation and candidate actions belong to the downstream action layer. These remain separate artifact contracts rather than hidden stages of a single report operation.

Claim status is preserved across transformer activation, Qwen attention, diffusion deformation, and boundary reports even when their mathematical origins differ.

The accompanying SOFRS v1.0 report collection and separate validator fixture are listed as Artifacts B15–B16.

---

13 Boundary

Paper XII claims:

1. SOF diagnostics provide a reproducible workflow for turning compatible sectorizations and observable families into SOF Reports.
2. Attention heads, activation patterns, expert-routing pairs, diffusion schedules, connectivity components, user/item clusters, control flags, Markov partitions, and other information decompositions can support white-box SOF diagnostics when the finite realization is explicit.
3. In selected case studies, SOF Reports expose cross-sector support, repair/freeze structure, trajectory direction, or failure status not visible from raw native coordinates alone.
4. SOF is deployable as a methodology because it can produce positive, negative, and degenerate reports with explicit claim status.
5. Under the Black-Box SOF Diagnostic Principle, API-only systems can support Behavioral SOF Reports using prompt protocols and task classes as probe sectors, provided measurable outputs exist and the report is labeled as a weak input–output diagnostic rather than a strict internal realization.

6. Under the Report Relativity Principle, a report is derived from a declared realization and reporting specification rather than directly from the source system; same-source reports need explicit alignment before comparison.

Paper XII does not claim:

1. a universal explainability theory;
2. that every system has a meaningful sectorization;
3. that SOF diagnostics are always better than native diagnostics;
4. that proxy observables determine $R_1/R_2/D$ without an additional bridge theorem;
5. that the reference implementations constitute a complete production software package;
6. that attention-head sectorization is unique or canonical for all transformers;
7. that black-box prompt protocols define projector-valued sectors or reveal internal LLM mechanisms;
8. that task-scoped groundedness is a universal hallucination detector, or that protocol repair is equivalent to Lie-depth D-repair;
9. that the candidate Behavioral Wall vocabulary is a universal wall taxonomy or a claim about hidden model mechanisms;
10. that recommender dead-zone reports replace online A/B testing or causal evaluation;
11. that dynamic-maze connectivity repair is fixed-sector Lie-depth D-repair;
12. that the dense four-expert top-2 MoE control implies that larger or production MoE routers have no frozen expert pairs.
13. a canonical alignment or difference operator between two SOF Reports;
14. that a recorded repair event is an instruction to modify the system;
15. an action policy, Action Set, or Action Algebra;
16. that envelope validity or protocol admission certifies the scientific adequacy of a domain-specific sectorization or observable family.

---

14 Conclusion

Paper XII turns SOF from an abstract observable language into a deployable diagnostic methodology. The unit of deployment is the SOF Report. Its job is to record how sectors were chosen, how observables were extracted, what support or repair structure was found, which wall or trajectory features appeared, and what claim-status boundary applies.

The unit of deployment is not the source system itself. The governing chain is

$$S \longrightarrow \mathcal{F}_\eta \longrightarrow \mathcal{R}_{\eta,\Theta_{\mathrm{rep}}}.$$

The source is realized before it is reported. The resulting report is a real, versioned protocol object, but it remains relative to the chosen sectors, observables, and reporting semantics.

The real pretrained Qwen audit illustrates why this matters. Attention heads already produce information sectorizations at multiple granularities. SOF does not need to invent those sectors; it needs to report them, audit them, and make their observable consequences comparable.

Here comparable means that the reports share a machine-readable diagnostic grammar. It does not mean that sectors, observables, normalizations, or bridge semantics are already aligned. That additional object is supplied only by the Paper XIII comparison language.

The black-box audit illustrates the complementary point. A system need not expose weights in order to support an observable report. Prompt protocols, task classes, response formats, and repair transitions can define a behavioral diagnostic layer, provided the report does not confuse interface behavior with internal mechanism. SOF is therefore an observable framework rather than a weight framework.

The MoE, maze, and recommender reports broaden the same methodology. Expert routing supplies natural sectors, a changing door topology supplies an immediately visible wall record, and a user–item propagation graph supplies a structural coverage report. Their native meanings differ, but the SOF Report grammar remains stable.

This also fixes the extension model for later applications. The SOF program maintains the versioned object and report language; each domain application is responsible for the scientific justification of its realization. New fields of application should therefore enter through declared sectorizations, observable families, native constraints, and failure boundaries rather than through unsupported analogies. The protocol can make such analyses inspectable and interoperable, but it cannot replace the domain knowledge required to construct them.

At the program level, Paper VIII defines the sectorized observable object, Paper IX supplies its deformation geometry, Paper X organizes the observable pipeline and Registry, Paper XI classifies its wall records and signatures, and Paper XII turns those structures into a versioned diagnostic artifact. The result is not a claim that the registered systems share one mechanism, but that they can be examined through one explicit report grammar that prepares artifacts for later aligned comparison.

This fixes the first layer of a three-part protocol stack:

Language	Paper	Operation
Report	XII	describe one system or run
Comparison	XIII	align two reports and compute Δ_{audit}
Action	XIV	interpret Δ_i before deriving candidate actions

The boundaries matter. A Report describes; an Audit compares; Action Semantics interprets the comparison before policy selection.

The methodological promise is therefore concise: **no weights required, only observables.**

Paper XII does not propose a new theory of neural networks, quantum systems, control, or stochastic processes. It proposes a common, versioned diagnostic reporting protocol for systems admitting compatible sectorizations or stable diagnostic probe partitions.

The present protocol is strongest at the discrete accessibility layer: it records support, bridges, frozen pairs, repairs, and observable wall events. Continuous response constants, word and Lie depths, and trajectory variation already provide quantitative seeds for a possible future **SOF Geometry**. Such a program could study strength, depth, transversality, persistence, and distance to observable walls. Paper XII neither defines that geometry nor claims a wall-stiffness invariant; it only establishes the reporting layer on which those quantities could later be declared and compared.

The VIII–XII sequence therefore begins with sectorized observable objects and ends with a deployable report: no common internal mechanism is required, only observable structure and an explicit claim boundary.

Appendix A SOF Report Specification v1.0

This appendix is normative. A SOFRS v1.0 artifact contains the version key `sofrs_version` and the eight diagnostic fields defined in Section 2. The controlled `claim_status` vocabulary supports machine-readable aggregation and prepares reports for later aligned comparison, while `claim_note` carries non-normative human qualification. Domain-specific metadata may be added without changing the eight-field report grammar. The executable schema is Artifact B1.

The JSON Schema below defines envelope validity only. Paper XII protocol admission additionally requires `report_id`, `system`, `claim_note`, an explicit failure boundary, and conditional Level III evaluator provenance. Those rules are encoded by Artifact B2; they do not mutate the frozen v1.0 envelope contract. Downstream protocol keys such as `reference`, `candidate`, `alignment`, `signature`, `comparison_role`, `transformation_contract`, `contract_evaluation`, `action_semantics`, `action_set`, and `selection` are reserved for downstream comparison and action artifacts and should not appear as top-level SOFRS fields.

The envelope validator (Artifact B3) also performs a schema-drift check: it extracts the JSON block below, parses it, and requires semantic equality with the canonical schema before validating any `.sofreport` artifact. Protocol admission is checked separately by Artifact B4. The schema `$id` is a stable namespace identifier rather than a network dependency; validation uses the versioned repository copy.


```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://rime-project.local/schemas/sofrs/v1.0.schema.json",
  "title": "SOF Report Specification (SOFRS) v1.0",
  "description": "Versioned machine-readable contract for reports emitted by the Paper XII SOF
   ↪  Diagnostic Protocol.",
  "type": "object",
  "required": [
    "sofrs_version",
    "sectorization",
    "observable_family",
    "support_matrix",
    "bridge_matrix",
    "repair_matrix",
    "wall_record",
    "claim_status",
    "failure_modes"
  ],
  "properties": {
    "sofrs_version": {
      "description": "SOF Report Specification version used by this artifact.",
      "const": "1.0"
    },
    "sectorization": {
      "description": "Sector origin, construction rule, dimensions or probe classes, and
       ↪  realization status.",
      "type": "object",
      "minProperties": 1
    },
    "observable_family": {
      "description": "Named structural, behavioral, failure, operator, kernel, transition, or
       ↪  registered analogue observables.",
      "type": "object",
```

```
      "minProperties": 1
    },
    "support_matrix": {
      "description": "Direct cross-sector support or a clearly labeled behavioral support
       ↪  analogue.",
      "type": ["object", "array", "null"]
    },
    "bridge_matrix": {
      "description": "Length-two, commutator, word-depth, routing, or protocol bridge data.",
      "type": ["object", "array", "null"]
    },
    "repair_matrix": {
      "description": "Frozen-to-active pairs or registered repair transitions with layer or step
       ↪  metadata.",
      "type": ["object", "array", "null"]
    },
    "wall_record": {
      "description": "Wall events and any trajectory summary associated with a supplied
       ↪  deformation path.",
      "type": ["object", "array", "null"]
    },
    "claim_status": {
      "description": "Controlled claim class for the report as a whole.",
      "enum": [
        "theorem",
        "evidence",
        "diagnostic",
        "proxy_only",
        "boundary",
        "failure",
        "negative_control"
      ]
    },
    "claim_note": {
      "description": "Optional human-readable qualification of the controlled claim status.",
      "type": "string"
    },
    "failure_modes": {
      "description": "Applicability warnings and known interpretation boundaries. Use an empty
       ↪  array only when none are known.",
      "type": "array",
      "items": {"type": ["object", "string"]},
      "uniqueItems": true
    }
  },
  "not": {
    "anyOf": [
      {"required": ["schema_version"]},
      {"required": ["repair_candidates"]},
      {"required": ["wall_records"]},
      {"required": ["trajectory_summary"]}
    ]
  },
  "additionalProperties": true
}
```

The schema forbids the superseded top-level names `schema_version`, `repair_candidates`, `wa ll_records`, and `trajectory_summary`. A trajectory summary, when present, is nested inside `w` all_record. Specification revisions that change required keys or controlled vocabularies must increment the SOFRS version rather than silently changing the meaning of v1.0.

Appendix B Computational Artifacts

The main text refers to audit roles rather than repository paths. This appendix records the exact reproducibility layer using short paths.

B.1 Contracts and Validators

The default directory in this table is schemas/sofrs/ for B1–B2 and experiments/paper12/ for B3–B4.

Artifact	Role in Paper XII	Short path
B1	frozen SOFRS v1.0 envelope schema	v1.0.schema.json
B2	Paper XII protocol-admission profile	paper12-protocol-profile-v1.0.json
B3	envelope and Appendix schema-drift validator	validate_sofreport.py
B4	stronger protocol-admission validator	validate_protocol_admission.py

B.2 Paper XII Audits

The default directory in this table is experiments/paper12/.

Artifact	Role in Paper XII	Short path
B5	transformer activation-sector audit	transformer_activation_sof.py
B6	transformer batch robustness sweep	transformer_batch_sweep.py
B7	revision-pinned Qwen attention audit	qwen_attention_sof.py
B8	MoE route-sector audit	moe_expert_sof.py
B9	bias-driven private-expert repair control	moe_bias_repair_sof.py
B10	diffusion-time denoising audit	diffusion_denoising_sof.py
B11	dynamic-maze wall crossing	maze_wall_crossing.py
B12	recommender coverage and dead-zone audit	recommender_sof.py
B13	behavioral/API-level language-model audit	blackbox_llm_sof.py
B14	multi-system boundary-fixture generator	failure_cases.py
B15	admitted reference reports	results/*.sofreport
B16	envelope-valid validator fixture	results/failure_cases.fixture.json

B.3 Cross-Program Support

The default directory in this table is experiments/.

Artifact	Role in Paper XII	Short path
B17	quantum gate-log T-blind control	quantum/quantum_gateset_t_blind_control.py
B18	quantum state-trajectory probe control	quantum/quantum_state_trajectory_sof.py
B19	Kalman, PDE, and combinatorial handoff	paper10/control_pde_combinatorial_sof.py
B20	barrier-finance handoff	paper10/barrier_option_sof.py

Bibliographic Notes

Program lineage. Paper VIII defines the SOF object layer; Paper IX defines SOF deformations and observable trajectories; Paper X defines the Universal Observable Pipeline and SOF Registry; Paper XI defines observable wall records and wall-spectrum features [3, 2, 4, 1]. Paper XII uses these layers as report fields.

Downstream protocol lineage. Paper XIII consumes two SOF Reports only after explicit sector and observable alignment and emits a `.sofaudit` comparison artifact. Paper XIV interprets the resulting signature coordinates before deriving a `.sofaction` candidate set. These later layers clarify the boundary of SOFRS; they are not operations performed by a single SOF Report.

Diagnostic precedents. The report orientation is analogous in spirit to test reports, coverage reports, profiling reports, static-analysis reports, model cards, and interpretability dashboards: the output is a structured artifact with claim-status metadata, not merely a positive example [9, 12].

Transformer and diffusion background. Attention-head analysis, activation pattern studies, expert-routing diagnostics, and diffusion denoising analyses provide external contexts in which sectorization-like decompositions arise natively. Paper XII uses these contexts diagnostically and does not claim a general theory of transformer or diffusion-model explainability [13, 5, 11, 8].

MoE routing background. DeepSeekMoE introduces fine-grained expert segmentation and shared experts; DeepSeek-V3 describes auxiliary-loss-free load balancing through routing-bias updates. Paper XII abstracts only the observable routing-repair pattern and does not audit DeepSeek parameters or claim a general MoE load-balancing theorem [6, 7].

References

[1] WuJun Chen. Observable classification theory for SOFs: Wall records, observable invariants, taxonomy, and smooth local models. *Preprint*, 2026. Paper XI of the RIME program. DOI: https://doi.org/10.5281/zenodo.21350768.

[2] WuJun Chen. Observable dynamics of sectorized observable frameworks: Deformations, rate separation, and observable dynamics. *Preprint*, 2026. Paper IX of the RIME program. DOI: https://doi.org/10.5281/zenodo.21287695.

[3] WuJun Chen. Sectorized observable framework: A sectorized observable architecture for spectral accessibility. *Preprint*, 2026. Paper VIII of the RIME program. DOI: https://doi.org/10.5281/zenodo.21287278.

[4] WuJun Chen. The universal observable pipeline for sectorized observable frameworks: SOF realization and cross-species registry evidence. *Preprint*, 2026. Paper X of the RIME program. DOI: https://doi.org/10.5281/zenodo.21288036.

[5] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT's attention, 2019. arXiv:1906.04341.

[6] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models, 2024. arXiv:2401.06066.

[7] DeepSeek-AI. DeepSeek-V3 technical report, 2024. arXiv:2412.19437.

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. arXiv:2006.11239.

[9] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

[10] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback, 2022. arXiv:2203.02155.

[11] Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.

[12] Inioluwa Deborah Raji, Andrew Smart, Rebecca N. White, Margaret Mitchell, Timnit Gebru, Ben Hutchinson, Jamila Smith-Loud, Daniel Theron, and Parker Barnes. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pages 33–44, 2020.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.